

Quick select for text field widget

A reusable Flutter dialog widget that allows users to edit the comprehensive insurance coverage (sum insured) for a vehicle. It supports manual input and quick-select preset amounts, and returns the updated value via a callback.

🌟 Features

- Displays vehicle information (name & registration number)
- Editable **Sum Insured** field with currency prefix
- Horizontal **Quick Select** chips for common values
- Auto-sync between manual input and quick selection
- Pre-selects a quick amount if it matches the initial value
- Clean, rounded dialog UI
- Returns updated value using a callback

🍁 Widget Overview

Type: `StatefulWidget`

Use case: Update vehicle comprehensive coverage in insurance flows

🦍 Constructor Parameters

Parameter	Type	Required	Description
<code>carName</code>	<code>String</code>	✓	Vehicle display name
<code>carReg</code>	<code>String</code>	✓	Vehicle registration number
<code>initialSumInsured</code>	<code>double</code>	✓	Initial coverage amount
<code>onUpdate</code>	<code>ValueChanged<double></code>	✓	Callback fired with updated value



Internal State

Variable	Type	Purpose
<code>_controller</code>	<code>TextEditingController</code>	Controls the sum insured input field
<code>_sumInsured</code>	<code>double</code>	Current selected/entered amount
<code>_selectedQuickAmountIndex</code>	<code>int?</code>	Tracks selected quick amount
<code>quickAmounts</code>	<code>List<double></code>	Preset selectable amounts

Key Behaviors

1. Initialization

- Sets initial sum insured
- Formats it into the text field
- Auto-selects matching quick amount (if any)

2. Quick Select

- Updates sum insured instantly
- Syncs input field
- Highlights selected chip

3. Manual Input

- Parses numeric input safely
- Clears quick selection on custom values

4. Update Action

- Calls `onUpdate(sumInsured)`
- Closes dialog



UI Structure

1. Title & description
2. Vehicle info card (icon + details)
3. Sum insured input field
4. Quick select chips (horizontal scroll)
5. Informational note
6. Cancel & Update buttons

Usage Example

```
showDialog(  
  context: context,  
  builder: (_) => EditComprehensiveCoverageDialog(  
    carName: 'Toyota Corolla',  
    carReg: 'KDA 123A',  
    initialSumInsured: 50000,  
    onUpdate: (newValue) {  
      print('Updated coverage: $newValue');  
    },  
  ),  
);
```

Customization Tips

- Modify `quickAmounts` to fit your pricing tiers
- Replace `CustomButton` with `ElevatedButton` if needed
- Add input formatters for currency formatting
- Add min/max validation before update

Notes

- Manual input clears quick selection automatically
- Quick select overrides text input
- Dialog height adapts to content (`MainAxisSize.min`)

Full Source Code

```
import 'package:flutter/material.dart';  
import 'package:nanduba/export.dart';  
  
class EditComprehensiveCoverageDialog extends StatefulWidget {  
  final String carName;  
  final String carReg;  
  final double initialSumInsured;  
  final ValueChanged<double> onUpdate;  
  
  const EditComprehensiveCoverageDialog({
```

```

        super.key,
        required this.carName,
        required this.carReg,
        required this.initialSumInsured,
        required this.onUpdate,
    });

    @override
    State<EditComprehensiveCoverageDialog> createState() =>
        _EditComprehensiveCoverageDialogState();
}

class _EditComprehensiveCoverageDialogState
    extends State<EditComprehensiveCoverageDialog> {
    late TextEditingController _controller;
    late double _sumInsured;
    int? _selectedQuickAmountIndex;

    final List<double> quickAmounts = [30000, 50000, 50000, 100000];

    @override
    void initState() {
        super.initState();
        _sumInsured = widget.initialSumInsured;
        _controller = TextEditingController(text: _sumInsured.toStringAsFixed(0));

        _selectedQuickAmountIndex =
            quickAmounts.indexWhere((amount) => amount == _sumInsured);
        if (_selectedQuickAmountIndex == -1) _selectedQuickAmountIndex = null;
    }

    void _onQuickSelect(int index) {
        setState(() {
            _selectedQuickAmountIndex = index;
            _sumInsured = quickAmounts[index];
            _controller.text = _sumInsured.toStringAsFixed(0);
        });
    }

    void _onInputChanged(String val) {
        final parsed =
            double.tryParse(val.replaceAll(',', '').replaceAll('\$', '').trim());
        if (parsed != null) {
            setState(() {
                _sumInsured = parsed;
                _selectedQuickAmountIndex = null;
            });
        }
    }
}

```

```

}

@override
Widget build(BuildContext context) {
  return Dialog(
    backgroundColor: Colors.white,
    insetPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 24),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    child: Padding(
      padding: const EdgeInsets.fromLTRB(20, 24, 20, 16),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Text(
            'Edit Comprehensive Coverage',
            style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
            textAlign: TextAlign.center,
          ),
          const SizedBox(height: 6),
          const Text(
            'Adjust coverage limits for this vehicle.',
            style: TextStyle(fontSize: 14, color: Colors.black54),
            textAlign: TextAlign.center,
          ),
          const SizedBox(height: 20),
          Container(
            padding: const EdgeInsets.all(12),
            decoration: BoxDecoration(
              color: Colors.grey.shade100,
              borderRadius: BorderRadius.circular(12),
            ),
            child: Row(
              children: [
                SvgPicture.asset(
                  AppSvgs.carIcon,
                  height: 25,
                  width: 25,
                  color: AppColors.primary,
                ),
                const SizedBox(width: 12),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        widget.carName,
                        style: const TextStyle(
                          fontWeight: FontWeight.w600, fontSize: 16),

```

```

        ),
        Text(
          'Reg: ${widget.carReg}',
          style: const TextStyle(
            fontSize: 12, color: Colors.black54),
        ),
      ],
    ),
  ),
  1,
),
const SizedBox(height: 20),
const Align(
  alignment: Alignment.centerLeft,
  child: Text(
    'Sum insured',
    style: TextStyle(fontSize: 14, fontWeight: FontWeight.w600),
  ),
),
const SizedBox(height: 8),
TextField(
  controller: _controller,
  keyboardType:
    const TextInputType.numberWithOptions(decimal: false),
  decoration: InputDecoration(
    prefixText: '\$ ',
    border:
      OutlineInputBorder(borderRadius: BorderRadius.circular(8)),
    contentPadding:
      const EdgeInsets.symmetric(horizontal: 12, vertical: 14),
  ),
  onChanged: _onInputChanged,
),
const SizedBox(height: 12),
const Align(
  alignment: Alignment.centerLeft,
  child: Text(
    'Quick select:',
    style: TextStyle(fontSize: 14, fontWeight: FontWeight.w600),
  ),
),
const SizedBox(height: 8),
SingleChildScrollView(
  scrollDirection: Axis.horizontal,
  child: Row(
    children: List.generate(quickAmounts.length, (index) {
      final amount = quickAmounts[index];

```

```

        final isSelected = _selectedQuickAmountIndex == index;
        return Padding(
          padding: const EdgeInsets.only(right: 8),
          child: ChoiceChip(
            label: Text('\${amount.toStringAsFixed(0)}'),
            selected: isSelected,
            onSelect: (selected) {
              if (selected) _onQuickSelect(index);
            },
            selectedColor: Colors.blue.shade700,
            labelStyle: TextStyle(
              color: isSelected ? Colors.white : Colors.black87,
              fontWeight: FontWeight.w600,
            ),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(8)),
          ),
        );
      },
    ),
  ),
  const SizedBox(height: 20),
  Row(
    children: [
      Expanded(
        child: CustomButton(
          label: 'Cancel',
          backgroundColor: Colors.white,
          textcolor: Colors.black87,
          borderColor: Colors.grey,
          height: 40,
          onPressed: () => Navigator.pop(context),
        ),
      ),
      const SizedBox(width: 12),
      Expanded(
        child: CustomButton(
          label: 'Update',
          backgroundColor: AppColors.primary,
          textcolor: Colors.white,
          height: 40,
          onPressed: () {
            widget.onUpdate(_sumInsured);
            Navigator.pop(context);
          },
        ),
      ),
    ],
  ),
],

```

```
        ),  
        1,  
        ),  
        ),  
        );  
    }  
}
```